

TD 5 : Conditions, Pool de thread

- condition variable • wait/notify • pool de thread

Q1 写 empty, full

Q2 将 em + full 放在 pop 和 push 里

Q3 加 mutex

```
template<typename T>
class Queue {
    T** tab;                // 队列存储数组
    const size_t allocsize; // 队列分配的存储空间大小
    size_t begin;           // 队列开始位置的索引
    size_t sz;              // 队列当前大小
    mutable std::mutex m;   // 互斥锁, 用于线程安全

    // 私有函数, 不带互斥锁保护
    bool empty() const {    // 检查队列是否为空
        return sz == 0;
    }
    bool full() const {     // 检查队列是否已满
        return sz == allocsize;
    }

public:
    // 构造函数
    Queue(size_t size) : allocsize(size), begin(0), sz(0) {
        tab = new T*[size]; // 分配存储空间
        memset(tab, 0, size * sizeof(T*)); // 初始化存储空间
    }
    // 获取队列当前大小
    size_t size() const {
        std::unique_lock<std::mutex> lg(m); // 加锁
        return sz;                          // 返回大小
    }
    // 弹出队列头元素
    T* pop() {
        std::unique_lock<std::mutex> lg(m); // 加锁
        if (empty()) {                    // 如果队列为空
            return nullptr;               // 返回空指针
        }
        auto ret = tab[begin];            // 获取头元素
        tab[begin] = nullptr;             // 将头部位置置为空
        sz--;                             // 队列大小减一
        begin = (begin + 1) % allocsize;  // 调整头部索引
        return ret;                       // 返回元素
    }
};
```

begin是队列的起始位置索引。

sz是当前队列中的元素数量。

allocsize是队列分配的总大小

```
// 向队列尾部插入元素
bool push(T* elt) {
    std::unique_lock<std::mutex> lg(m); // 加锁
    if (full()) {                       // 如果队列已满
        return false;                  // 返回false
    }
    tab[(begin + sz) % allocsize] = elt; // 插入元素
    sz++;                              // 队列大小加一
    return true;                       // 插入成功, 返回true
}

// 析构函数
~Queue() {
    // 注释说明: 看似无用的析构代码, 如果还在使用队列则不应销毁
    for (size_t i = 0; i < sz; i++) { // 遍历队列
        auto ind = (begin + i) % allocsize; // 计算索引
        delete tab[ind];              // 删除元素
    }
    delete[] tab;                    // 删除队列存储空间
}
```

Q4 条件变量来同步线程的操作, 使得当队列为空时, 尝试pop的线程会等待, 而当队列满时, 尝试push的线程会等待

```
class Queue {
    T **tab;                // 指针数组, 用于存储队列中的元素
    const size_t allocsize; // 队列的最大容量
    size_t begin;           // 队列头部元素的索引
    size_t sz;              // 队列当前的大小
    mutable std::mutex m;   // 互斥锁, 用于保护队列
    std::condition_variable cv; // 条件变量, 用于阻塞和唤醒线程

    // 省略其余部分 ...
};

T* pop() {
    std::unique_lock<std::mutex> lck(m); // 创建一个互斥锁保护的锁定区域
    while (empty()) {                   // 当队列为空时, 等待
        cv.wait(lck);                  // 等待条件变量, 释放锁, 当被唤醒时重新锁定
    }
    // 如果队列满了, 则通知其他可能正在等待的push线程
    if (full()) {
        cv.notify_all();               // 唤醒所有等待条件变量的线程
    }
    auto ret = tab[begin];              // 取出队列头部的元素
    tab[begin] = nullptr;               // 将取出元素的位置置为空
    sz--;                               // 队列大小减1
    begin = (begin + 1) % allocsize;    // 队列头部索引循环移动
    return ret;                        // 返回取出的元素
}

bool push(T* elt) {
    std::unique_lock<std::mutex> lg(m); // 创建一个互斥锁保护的锁定区域
    while (full()) {                   // 当队列满时, 等待
        cv.wait(lg);                  // 等待条件变量, 释放锁, 当被唤醒时重新锁定
    }
    // 如果队列为空了, 则通知其他可能正在等待的pop线程
    if (empty()) {
        cv.notify_all();              // 唤醒所有等待条件变量的线程
    }
    tab[(begin + sz) % allocsize] = elt; // 在队列尾部插入新元素
    sz++;                              // 队列大小加1
    return true;                      // 插入成功, 返回true
}
```

Q5 Malgré le fait qu'il puisse y avoir des Threads bloqués en wait sur la condition, d'autres threads peuvent quand même acquérir le lock. Expliquez pourquoi. 为什么即使有线程因为等待某个条件而阻塞, 其他线程仍然可以获取相同的锁。

wait fait de façon atomique un unlock avant de dormir.

Au réveil il reprend le lock atomiquement.

Donc on dort sur wait, mais SANS le lock à la main

当线程在等待某个条件 (比如等待队列有数据可以取出) 时, 它会做两件事情:

放下锁: 它会释放它所持有的锁, 这样其他线程就可以使用这个锁来添加数据到队列中。

进入等待状态: 线程进入一种“休眠”状态, 不会进行任何操作, 也不会占用任何资源。

当条件满足了 (比如其他线程放入了数据), 等待的线程会被唤醒, 并且在继续执行之前:

自动拿回锁: 它会自动尝试重新获取之前释放的锁。

1.1.3 Discussion, variantes Q 6

在一个队列实现中使用两个不同的条件变量的理由

在多线程环境中，使用两个条件变量可以提高效率，因为它允许我们分别通知等待队列为空的消费者线程和等待队列不满的生产者线程。如果只用一个条件变量，每次队列状态改变时所有等待的线程都会被唤醒，无论它们是等待队列变满还是变空，这会导致不必要的唤醒和锁竞争，降低程序效率

Alors l'intérêt en principe on ne notifie que des gens intéressés, i.e. pas les producteurs si on push, pas les consommateurs si on pop.
MAIS, pas clair ici que l'on puisse avoir au même instant des consommateurs et des producteurs simultanément bloqués. Les deux conditions sont réellement exclusives l'une de l'autre. Donc pas d'intérêt pratique.

On fait quand même l'exercice demandé.

Même avec plusieurs conditions dans la classe, on veut a priori toutes les contrôler avec le même mutex, ça ne pose pas de problème et c'est nécessaire de protéger la condition bool sur laquelle le wait porte.

Il faut passer un unique_lock, on bascule donc là dessus si on a fait lock à la main" plus haut.

```
class Queue {
    T ** tab;
    const size_t allocsize;
    size_t begin;
    size_t sz;
    mutable std::mutex m;
    std::condition_variable cv_plein;
    std::condition_variable cv_vide;
```

```
T* pop() {
    std::unique_lock<std::mutex> lck(m); // 加锁
    while (empty()) { // 队列空则等待
        cv_vide.wait(lck); // 等待队列非空
    }
    // 此时队列已经非空，可以继续执行弹出操作
    if (full()) { // 如果队列之前满了，现在不满，通知生产者
        cv_plein.notify_all(); // 唤醒所有等待队列不满的生产者线程
    }
    auto ret = tab[begin]; // 取出队列头部元素
    tab[begin] = nullptr; // 将取出元素的位置设置为nullptr
    sz--; // 更新队列大小
    begin = (begin + 1) % allocsize; // 循环队列头部索引更新
    return ret; // 返回取出的元素
}

bool push(T* elt) {
    std::unique_lock<std::mutex> lg(m); // 加锁
    while (full()) { // 队列满则等待
        cv_plein.wait(lg); // 等待队列不满
    }
    // 此时队列已经不满，可以继续执行插入操作
    if (empty()) { // 如果队列之前空了，现在不空，通知消费者
        cv_vide.notify_all(); // 唤醒所有等待队列非空的消费者线程
    }
    tab[(begin + sz) % allocsize] = elt; // 在队列尾部插入新元素
    sz++; // 更新队列大小
    return true; // 插入成功，返回true
}
```

Q7 notify_one ou all

"Donc notify_one ne réveille qu'un seul thread bloqué."

notify_one只会唤醒一个正在等待的线程，而不是所有等待的线程。

"Par rapport à notify_all, Si tous les threads qui attendent en face sont identiques/symétriques/peuvent traiter la notification, c'est ok en général."

如果所有在等待的线程都在等待相同的条件，例如，它们都是消费者等待队列为空，那么使用notify_one通常是可以的，因为任何一个被唤醒的线程都可以处理通知。

"Par contre, si la nature des threads en attente est différente, le plus souvent notify one n'est pas une bonne idée."

然而，如果等待的线程是混合的，即有些是生产者等待插入（队列不满），有些是消费者等待取出（队列不空），那么notify_one通常不是一个好选择，因为它可能唤醒了一个错误类型的线程，导致它立刻又进入等待状态。

"Sur la version à une seule condition, on doit immédiatement se méfier."

在只有一个条件变量的队列版本中，你需要小心使用notify_one，因为这可能会导致错误类型的线程被唤醒。

"Ici, sur la version à deux conditions donc, ça paraît ok, on a soit bloqués sur empty, soit un paquet de producteurs bloqués sur full, mais jamais de mélange."

在有两个条件变量的版本中，这种情况看起来是可以接受的，因为你有一组线程只等待队列非空，另一组线程只等待队列不满，它们不会混在一起等待。

"Par contre si on adopte cette stratégie notify_one, il est essentiel de notifier à chaque pop/push, pas seulement quand l'état change."

但是，如果你选择使用notify_one策略，那么在每次pop或push操作时都需要发出通知，而不仅仅是在队列状态发生改变时

Question 8. On propose pour être plus ecace de faire le wait du pop dans un test if (empty()) cv.wait(m) ;. Que penser de cette solution non standard ?

一个可能的不好的场景是：消费者线程（C0）在队列为空时尝试pop并阻塞，生产者线程（P0）随后push一个元素并唤醒消费者线程（C0），然后另一个消费者线程（C1）尝试pop但此时队列为空。如果C0在醒来后不检查队列是否仍为空就尝试继续pop，它可能会错误地假设队列有元

On fait un if (cond) cv.wait(m) ; c'est bad.

Mauvaise idée. Déjà la question le dit c'est non standard, il doit y avoir une raison...

Donc un scenario bien foireux :

C0 pop sur vide (bloque), P0 push (notify C0), C1 pop (rien), C0 nit son pop c'est vide ! Morale : ne pas trop rééchir, suivre les schémas de synchro bien tracés.

Toujours faire un while ou la version lambda de wait.

Il faut un beau while, c'est le cas général, le cas notify all qui foire s'appelle "spurious wake", il est traité simplement par un while.

1.1.4 Terminaison 29

```
class Queue {
    T **tab; // 指针数组, 用于存储队列中的元素
    const size_t allocsize; // 队列的最大容量
    size_t begin; // 队列头部元素的索引
    size_t sz; // 队列当前的大小
    bool isBlocking; // 新增成员变量, 控制队列是否阻塞
    mutable std::mutex m; // 互斥锁, 用于保护队列
    std::condition_variable cv; // 条件变量, 用于阻塞和唤醒线程
};

void setBlocking(bool isBlocking) {
    std::unique_lock<std::mutex> lck(m);
    this->isBlocking = isBlocking;
    cv.notify_all(); // 唤醒所有等待的线程
}

T* pop() {
    std::unique_lock<std::mutex> lck(m);
    while (empty() && isBlocking) { // 如果队列为空并且处于阻塞模式, 则等待
        cv.wait(lck);
    }
    if (empty()) {
        return nullptr; // 如果队列为空且非阻塞模式, 立即返回nullptr
    }
    // 如果队列满了, 则通知其他可能正在等待的push线程
    if (full()) {
        cv.notify_all(); // 唤醒所有等待条件变量的线程
    }
    auto ret = tab[begin]; // 取出队列头部的元素
    tab[begin] = nullptr; // 将取出元素的位置置为空
    sz--; // 队列大小减1
    begin = (begin + 1) % allocsize; // 队列头部索引循环移动
    return ret; // 返回取出的元素
}

bool push(T* elt) {
    std::unique_lock<std::mutex> lg(m);
    while (full() && isBlocking) { // 如果队列满并且处于阻塞模式, 则等待
        cv.wait(lg);
    }
    if (full()) {
        return false; // 如果队列满且非阻塞模式, 立即返回false
    }
    // 如果队列为空了, 则通知其他可能正在等待的pop线程
    if (empty()) {
        cv.notify_all(); // 唤醒所有等待条件变量的线程
    }
    tab[(begin + sz) % allocsize] = elt; // 在队列尾部插入新元素
    sz++; // 队列大小加1
    return true; // 插入成功, 返回true
}
```

当程序要结束时, 可以通过调用setBlocking(false)来确保所有线程都能退出阻塞状态, 防止它们永远挂起而无法结束。

1.2 Pool de thread

```
#pragma once // 防止头文件重复包含
```

```
class Job {
public:
    virtual void run() = 0; // 纯虚函数, 需要在子类中实现具体的任务
    virtual ~Job() {}; // 虚析构函数, 确保子类能够正确析构
};
```

SleepJob类实现了Job类的run函数, 其中包含了具体的任务逻辑。此代码可以在一个线程池管理系统中使用, 其中Job对象可以被放入一个队列, 并由线程池中的线程逐一取出并执行

```
#include "Job.h"
#include <iostream>
#include <thread> // 包含thread库以使用线程功能

class SleepJob : public Job {
    int calcul(int v) {
        std::this_thread::sleep_for(std::chrono::seconds(1)); // 模拟长时间运行的任务, 这里是暂停1秒
        return v * 255; // 假设的计算, 将输入的整数乘以255
    }

    int arg; // 存储任务的输入参数
    int* ret; // 存储指向结果的指针

public:
    SleepJob(int arg, int* ret) : arg(arg), ret(ret) {} // 构造函数, 初始化输入参数和结果指针

    void run() override {
        std::cout << "Computing for arg = " << arg << std::endl; // 打印正在计算的信息
        *ret = calcul(arg); // 执行计算, 并将结果存储在ret指向的位置
        std::cout << "Obtained for arg = " << arg << " result = " << *ret << std::endl; // 打印获得的结果
    }

    ~SleepJob(){} // 析构函数
};

#include "SleepJob.h"

int main() {
    int ret; // 存储计算结果
    Job* pjob = new SleepJob(42, &ret); // 创建一个SleepJob任务, 输入参数为42, 结果存储在ret变量中
    pjob->run(); // 执行任务
    std::cout << ret << std::endl; // 打印任务结果

    delete pjob; // 清理分配的内存
    return 0;
}
```

1.2.2 Une class pool

线程池的一个实现。这个类包含两个主要的成员: 一个任务队列和一个线程向量。线程池的主要作用是提高并发执行任务的效率, 通过重用固定数量的线程来避免频繁地创建和销毁线程的开销。

```
// 防止头文件被多次包含
#ifndef SRC_POOL_H_
#define SRC_POOL_H_

#include "QueueBlock.h" // 包含自定义的阻塞队列头文件
#include "Job.h" // 包含任务类的头文件
#include <vector> // 包含标准模板库中的向量容器
#include <thread> // 包含线程相关功能

// 声明一个命名空间
namespace pr {

    // 工作线程函数, 负责从队列中取出任务并执行
    void poolWorker(Queue<Job*> queue) {
        while (true) { // 无限循环, 不断地执行任务
            Job* j = queue->pop(); // 从队列中取出一个任务
            if (j == nullptr) {
                // 如果从队列中取出的任务为空, 则退出循环
                return;
            }
            j->run(); // 执行取出的任务
            delete j; // 删除任务, 避免内存泄漏
        }
    }
}
```

- extraire un Job de la queue
- le traiter

Pool类的start方法用于启动线程池, stop方法用于停止线程池并等待所有工作线程结束。submit方法用于将新的任务提交给线程池。poolWorker函数是每个工作线程执行的函数, 它从任务队列中循环取任务执行, 直到取到nullptr为止, 这通常是线程池关闭的信号。

```
// 线程池类
class Pool {
    Queue<Job> queue; // 任务队列
    std::vector<std::thread> threads; // 存储所有工作线程的向量

public:
    // 构造函数，初始化任务队列的大小
    Pool(int qsize) : queue(qsize) {}

    // 启动线程池，创建指定数量的工作线程
    void start(int nbthread) {
        threads.reserve(nbthread); // 预留空间以避免重复分配
        for (int i=0; i < nbthread; i++) {
            // 创建新的工作线程并加入到线程向量中
            threads.emplace_back(poolWorker, &queue);
        }
        // 在threads向量的末尾添加一个新的线程，这个线程将执行
        // poolWorker函数，并且有权访问queue任务队列
    }
};
```

```
// 停止线程池的运行
void stop() {
    queue.setBlocking(false); // 设置队列为非阻塞模式
    for (auto &t : threads) {
        // 等待所有工作线程结束
        t.join();
    }
    threads.clear(); // 清空工作线程向量
}

// 析构函数
~Pool() {
    stop(); // 调用stop以确保所有线程都被正确清理
}

// 提交任务到线程池
void submit(Job *job) {
    queue.push(job); // 将任务加入到队列中
}

};

// namespace pr

#endif /* SRC_POOL_H_ */
```

1.2.3 progression du calcul

当线程池中的多个线程在执行任务时，Barrier可以确保所有的任务都完成后再继续执行一些操作

```
#ifndef __BARRIER_H_
#define __BARRIER_H_

#include <mutex>
#include <condition_variable>

// Barrier类用于线程同步，等待一定数量的事件发生后再继续执行
class Barrier {
    int counter; // 当前已完成的事件数量
    const int MAX; // 需要等待的事件总数
    std::mutex m; // 互斥锁，用于保护共享资源
    std::condition_variable cv; // 条件变量，用于阻塞和唤醒线程

public:
    // 构造函数，接收一个整数max作为需要等待的事件总数
    Barrier(int max) : counter(0), MAX(max) {}

    // done函数，每次调用都会增加counter计数，当达到MAX时通知所有等待的线程
    void done() {
        std::unique_lock<std::mutex> l(m); // 获取互斥锁
        counter++; // 增加计数
        if (counter == MAX) { // 当计数达到MAX时
            cv.notify_all(); // 唤醒所有等待的线程
        }
    }

    // waitFor函数，调用此函数的线程会等待直到事件总数达到MAX
    void waitFor() {
        std::unique_lock<std::mutex> l(m); // 获取互斥锁
        while (counter != MAX) { // 如果当前计数还未达到MAX
            cv.wait(l); // 阻塞线程，直到其他线程调用done()函数
        }
    }
};

#endif /* __BARRIER_H_ */
```

```
#include "Pool.h" // 包含线程池的头文件
#include "Barrier.h" // 包含Barrier类的头文件
#include "Job.h" // 包含Job类的头文件
#include <iostream> // 包含标准输入输出流的头文件
#include <thread> // 包含线程相关功能的头文件
#include <vector> // 包含向量容器的头文件

using namespace std;
using namespace pr;

class SleepJobBarrier : public Job {
    int calcul(int v) { // 模拟计算函数
        cout << "Computing for arg =" << v << endl;
        this_thread::sleep_for(1s); // 模拟耗时操作
        int ret = v * 255; // 模拟计算结果
        cout << "Obtained for arg =" << arg << " result " << ret << endl;
        return ret;
    }

    int arg; // 任务参数
    Barrier* barrier; // 指向Barrier对象的指针
    int* ret; // 指向结果的指针

public:
    SleepJobBarrier(int arg, Barrier* bar, int* ret) : arg(arg), barrier(bar), ret(ret) {}

    void run() override {
        *ret = calcul(arg); // 执行计算并保存结果
        barrier->done(); // 通知barrier计算完成
    }

    ~SleepJobBarrier() {}
};

int main() {
    const int NBTHREAD = 20; // 线程池中线程的数量
    const int NBJOB = 60; // 要处理的任务数量

    Pool pool(15); // 创建一个大小为15的线程池 任务队列
    vector<int> results(NBJOB); // 存储任务结果的向量

    pool.start(NBTHREAD); // 启动线程池

    Barrier b(NBJOB); // 创建一个Barrier，等待NBJOB数量的任务完成
    for (int i = 0; i < NBJOB; i++) {
        // 提交任务到线程池
        pool.submit(new SleepJobBarrier(i, &b, &results[i]));
    }

    b.waitFor(); // 等待所有任务完成
    pool.stop(); // 停止线程池

    for (int &i : results) {
        // 输出所有任务的结果
        cout << i << " ";
    }
    cout << endl;

    return 0;
}
```

